

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 02 -

EXPERIMENTAÇÃO E MVP DE MODELOS

SUMÁRIO

O QUE VEM POR AÍ?3

HANDS ON4

SAIBA MAIS.....5

MERCADO, CASES E TENDÊNCIAS34

O QUE VOCÊ VIU NESTA AULA?36

REFERÊNCIAS.....38



O QUE VEM POR AÍ?

Imagine começar um projeto de Machine Learning e, logo de cara, se deparar com furos nos dados ou valores que fogem completamente do padrão esperado, como registros de pacientes com dados de saúde inconsistentes ou resultados de exames implausíveis. Como garantir que o modelo final seja confiável? Nesta aula, vamos explorar exatamente isso: as etapas iniciais cruciais de um projeto de Machine Learning, desde o mapeamento dos dados disponíveis até a análise exploratória (Exploratory Data Analysis – EDA) e a construção de um MVP de modelo.

Veremos como inspecionar os dados brutos em busca de padrões e anomalias, identificar outliers (valores atípicos) e dados faltantes, além de aplicar técnicas iniciais de preparação que melhoram a qualidade do conjunto de dados. Tudo isso de forma leve e prática, usando um exemplo clássico para a classificação de Machine Learning – o conjunto de dados de Doença Cardíaca (Heart Disease) – para demonstrar na prática como as experimentações iniciais pavimentam o caminho para modelos bem-sucedidos.

O dataset de Doença Cardíaca (Heart Disease) disponível no repositório da UCI (vide referências) é um conjunto de dados muito utilizado para problemas de classificação. Ele contém atributos relacionados a resultados de exames médicos e condições de saúde de pacientes (como idade, sexo, tipo de dor no peito, resultados de eletrocardiograma etc.). O objetivo primário é classificar corretamente o paciente entre as classes: Doença Cardíaca Presente (target > 0) ou Doença Cardíaca Ausente (target = 0), com base nessas características. No nosso curso, usaremos o dataset de Doença Cardíaca para entender EDA, preparação de dados e construção de um MVP.

Ao final desta aula, você entenderá por que “dados ruins levam a análises ruins” e como um MVP (Produto Mínimo Viável) de modelo pode ser usado para validar rapidamente uma ideia sem desperdiçar esforço. Em suma, vamos “quebrar o gelo” no processo de ciência de dados: antes de construir modelos complexos, é fundamental dissecar e entender os dados a fundo (o perfil de saúde e os resultados de exames dos pacientes), corrigir problemas evidentes e, só então, partir para experimentações de modelagem iniciais.

HANDS ON

Agora é hora de colocar a mão na massa! Nesta seção, vamos aplicar os conceitos de Análise Exploratória de Dados (EDA) e construção de MVP de modelo. Usaremos o dataset clássico de Doença Cardíaca (Heart Disease) do repositório UCI, como mencionado na introdução, para prever a presença ou ausência de doença cardíaca.



SAIBA MAIS

Uso Do Python E Bibliotecas

Para a execução do Hands On (na seção anterior), o Python se apresenta como a ferramenta ideal, dada a sua vasta coleção de bibliotecas dedicadas à Ciência de Dados e Machine Learning. A escolha dessas bibliotecas é estratégica para garantir a eficiência e a reprodutibilidade do trabalho, desde a manipulação inicial dos dados até a avaliação do modelo. As bibliotecas que utilizaremos são:

- **Pandas:** essencial para a manipulação e análise de dados. Ele oferece estruturas de dados rápidas e eficientes, como DataFrames e Series, que facilitam o carregamento, a inspeção e a preparação dos dados (como o tratamento de valores faltantes e a codificação de variáveis).
- **NumPy:** base para computação numérica em Python. Fornece suporte para arrays e matrizes grandes, com funções matemáticas de alto nível. O Pandas é construído sobre o NumPy.
- **Matplotlib e Seaborn:** bibliotecas padrão para visualização de dados. O Matplotlib é a base para a criação de gráficos estáticos, interativos e animados, enquanto o Seaborn oferece uma interface de alto nível mais focada em estatísticas, facilitando a criação de visualizações informativas (como boxplots, histogramas e heatmaps de correlação) essenciais para a fase de EDA.
- **Scikit-learn:** a biblioteca mais popular de Machine Learning em Python. É utilizada para modelagem, fornecendo algoritmos eficientes (como a Regressão Logística que usaremos para o MVP), ferramentas de pré-processamento (como o StandardScaler) e métricas de avaliação (accuracy_score, confusion_matrix, etc.).

É fundamental que essas bibliotecas estejam instaladas no seu ambiente de trabalho (Jupyter, Google Colab, Databricks Free Edition, ou outro de sua escolha).

Passo 1: Carregamento e Entendimento Inicial dos Dados

O primeiro passo é carregar os dados e fazer a inspeção inicial, alinhado à fase de Data Understanding do CRISP-DM.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Código-fonte 1 - Importar Bibliotecas (Python)
Fonte: Elaborado pelo autor (2026)

Baixe o arquivo "heart_disease_uci.csv" do Kaggle baseado no dataset de Heart Disease disponibilizado pelo UCI. Note que ele pode não ter cabeçalho. O arquivo e códigos utilizados também serão disponibilizados em um repositório do Github para apoio ao aluno.

```
path = "data/heart_disease_uci.csv"
# Carregar dados
df = pd.read_csv(path)
```

Código-fonte 2 - Carregar o Dataset (Python)
Fonte: Elaborado pelo autor (2026)

```
print(f"Shape dos dados: {df.shape}")
print(f"\nPrimeiras linhas do dataset:")
df.head(10)
```

Código-fonte 3 - Inspeção Inicial (Python)
Fonte: Elaborado pelo autor (2026)

O dataset carregado contém informações clínicas de pacientes para análise de doenças cardíacas. Cada linha representa um paciente, com variáveis como idade, sexo, tipo de dor no peito, pressão arterial, colesterol, resultados de exames e o diagnóstico (coluna `num`). Algumas colunas apresentam valores ausentes, que serão tratados nas etapas seguintes. O objetivo é prever a presença de doenças cardíacas a partir dessas variáveis, utilizando técnicas de análise exploratória e modelagem preditiva.

id	age	sex	dataset	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope	ca	thal	num
1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	150.0	False	2.3	downsloping	0.0	fixed defect	0
2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	108.0	True	1.5	flat	3.0	normal	2
3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	129.0	True	2.6	flat	2.0	reversable defect	1
4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	187.0	False	3.5	downsloping	0.0	normal	0
5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	172.0	False	1.4	upsloping	0.0	normal	0

Figura 1 – Dataset carregado
 Fonte: Elaborado pelo autor (2026)

Dicionário de Dados (Data Dictionary)

Para garantir o entendimento completo dos dados clínicos de pacientes para análise de doenças cardíacas (Heart Disease), utilizamos também um dicionário de dados descrevendo cada uma das colunas do dataset:

- **id**: identificador do paciente.
- **dataset**: origem do caso (Cleveland, Hungarian, Switzerland, VA Long Beach).
- **age**: idade (anos).
- **sex**: sexo (Male/Female).
- **cp**: tipo de dor no peito (typical angina, atypical angina, non-anginal, asymptomatic).
- **trestbps**: pressão arterial em repouso (mmHg).
- **chol**: colesterol sérico (mg/dl).
- **fbs**: glicemia em jejum > 120 mg/dl (True/False).
- **restecg**: ECG em repouso (normal, st-t abnormality, lv hypertrophy).
- **thalch**: frequência cardíaca máxima alcançada.
- **exang**: angina induzida por exercício (True/False).
- **oldpeak**: depressão do ST induzida por exercício (unidades “ST depression”).
- **slope**: inclinação do segmento ST de pico (upsloping, flat, downsloping).

- **ca**: número de vasos principais coloridos por fluoroscopia (0–3).
- **thal**: estado talassêmico (normal, fixed defect, reversable defect).
- **num**: diagnóstico (0 = sem doença; 1–4 = presença).

Informações gerais do dataset

No fluxo de trabalho de Ciência de Dados, após o carregamento bem-sucedido dos dados, o próximo passo é realizar uma inspeção inicial minuciosa para entender a estrutura e as características estatísticas do DataFrame recém-criado. Esta etapa é o alicerce da Análise Exploratória de Dados (EDA) e nos permite identificar de imediato o tipo de dados contidos em cada coluna, a presença e a extensão de dados faltantes e as primeiras métricas de distribuição.

Para isso, os métodos `df.info()` e `df.describe()` podem ser usados. Eles fornecem a primeira "fotografia" do conjunto de dados, guiando todas as decisões subsequentes de preparação e modelagem. Vamos explorar como cada um desses métodos contribui para o nosso entendimento inicial dos dados de Doença Cardíaca:

```
# Informações gerais sobre o dataset
print("=== INFORMAÇÕES GERAIS DO DATASET ===\n")
print(df.info())
print("\n=== ESTATÍSTICAS DESCRITIVAS ===\n")
df.describe()
```

Código-fonte 4 – Informações gerais sobre o dataset com `df.info()` e `df.describe()` (Python)
Fonte: Elaborado pelo autor (2026)

O método `df.info()` é importante para a inspeção estrutural do DataFrame. Ele retorna um resumo conciso do conjunto de dados, o que é fundamental na fase inicial de Data Understanding (Entendimento dos Dados):

- **Tipos de Dados (Dtypes)**: informa o tipo de dado de cada coluna (e.g., `int64`, `float64`, `object`). Isso é crucial para a preparação de dados, pois modelos de ML só aceitam entradas numéricas e colunas do tipo `object` (geralmente strings ou categorias) precisam ser codificadas.

- **Contagem de Valores Não-Nulos:** mostra quantos valores não-nulos existem em cada coluna. A diferença entre o total de entradas e o número de não-nulos indica a presença de dados faltantes (missing data), o que direciona a necessidade de imputação ou remoção (como faremos na etapa 3: Preparação e Limpeza de Dados).
- **Uso de Memória:** indica a quantidade de memória RAM utilizada pelo DataFrame, útil para trabalhar com grandes volumes de dados.

O método `df.describe()` é utilizado para obter um resumo estatístico descritivo das colunas, o que é a base para a Análise Exploratória de Dados (EDA) Univariada.

Para Variáveis Numéricas:

- **Tendência Central:** fornece a média e a mediana (através do 50% ou segundo quartil). A diferença significativa entre média e mediana sugere assimetria na distribuição (o que pode indicar a presença de outliers ou a necessidade de transformação de dados).
- **Dispersão:** apresenta o desvio-padrão (std), que mede o quão dispersos os dados estão em relação à média.
- **Range e Limites:** mostra os valores mínimo e máximo (útil para identificar valores implausíveis ou erros de registro, como uma idade de 200 anos ou um colesterol de 0, mencionados na discussão sobre outliers).
- **Quartis:** indica os quartis (25%, 50%, 75%), que são essenciais para construir boxplots e identificar outliers usando a regra do Intervalo Interquartil (IQR).

Para Variáveis Categóricas (quando `include='all'`):

- **Contagem (count):** total de entradas não-nulas.
- **Categoria Mais Frequente (top):** informa a categoria mais comum.
- **Frequência da Categoria Mais Frequente (freq):** indica quantas vezes a categoria mais frequente aparece (útil para avaliar o balanceamento das categorias).

- **Número de Valores Únicos (unique):** essencial para verificar a cardinalidade das variáveis categóricas (se há muitas categorias, pode ser necessário agrupá-las ou usar técnicas de codificação mais robustas).

Em resumo, `df.info()` e `df.describe()` são os primeiros comandos a serem executados em qualquer projeto de Ciência de Dados, pois fornecem uma fotografia inicial da estrutura, dos tipos e da qualidade estatística dos dados, pavimentando o caminho para a preparação e modelagem subsequentes.

Passo 2: Análise Exploratória de Dados (EDA)

A etapa de Análise Exploratória de Dados (EDA) é o coração do processo de Data Understanding. Como vimos, ela é essencial para que os dados "contem sua história" antes de partirmos para a modelagem preditiva. Nesta fase, faremos um mergulho mais profundo no dataset de Doença Cardíaca para ir além da inspeção inicial.

Focaremos em quatro pilares cruciais que impactarão diretamente a qualidade e a performance do nosso MVP: a Análise de Dados Faltantes (Missing Values), a inspeção da Distribuição da Variável Target (o diagnóstico de doença cardíaca, que é a variável que queremos prever), a Identificação de Outliers (valores atípicos que podem distorcer o treinamento do modelo) e a busca por Valores Inválidos ou Inconsistentes (erros de registro ou valores implausíveis no contexto médico). O objetivo é gerar os insights necessários para guiar as decisões de limpeza e preparação do próximo passo.

A Análise de Dados Faltantes é o ponto de partida da nossa Análise Exploratória de Dados (EDA). A presença de valores ausentes (frequentemente representados por NaN, Null ou outros caracteres) é um desafio comum e essencial nesta fase, pois a maioria dos modelos de Machine Learning não consegue processar entradas incompletas.

```
# Análise de valores ausentes
print("=== ANÁLISE DE MISSING VALUES ===\n")
missing_values = pd.DataFrame({
    'Coluna': df.columns,
    'Missing_Count': df.isnull().sum(),
    'Missing_Percentage': (df.isnull().sum() / len(df) *
100).round(2)
})
```

```
missing_values                                     =  
missing_values[missing_values['Missing_Count']]      >  
0].sort_values(  
    by='Missing_Percentage', ascending=False  
)
```

Código-fonte 5 – Análise de Missing Values (Python)
Fonte: Elaborado pelo autor (2026)

O bloco de código tem como objetivo central quantificar e calcular a porcentagem de valores ausentes (nulos) em cada coluna do DataFrame de Doença Cardíaca. Em resumo, ele está:

- **Identificando** quais colunas possuem valores NaN (nulos).
- **Somando** o total de valores ausentes por coluna (Missing_Count).
- **Calculando** a proporção desses ausentes em relação ao total de registros (Missing_Percentage).
- **Organizando** o resultado em um novo DataFrame (missing_values) e exibindo apenas as colunas que possuem dados faltantes, ordenadas da maior para a menor porcentagem.

É altamente recomendável visualizar a distribuição dos dados faltantes para obter uma compreensão mais clara da sua extensão e padrão. Um gráfico de barras ou um heatmap (mapa de calor) pode ilustrar de forma concisa quais colunas são mais afetadas e qual é a proporção de ausentes em relação ao total.

A seguir, exibiremos uma tabela com os dados faltantes por coluna e construiremos um gráfico de barras simples utilizando a biblioteca Matplotlib para visualizar a contagem de valores ausentes por coluna, facilitando a identificação imediata das features que mais exigirão estratégias de imputação ou tratamento.

```
if len(missing_values) > 0:  
    print(missing_values)  
  
    # Visualizar missing values  
    plt.figure(figsize=(10, 6))  
    plt.barh(missing_values['Coluna'],  
missing_values['Missing_Percentage'], color='coral')  
    plt.xlabel('Porcentagem de Missing Values (%)')  
    plt.title('Distribuição de Missing Values por Coluna')  
    plt.tight_layout()
```

```
plt.show()
else:
    print("Nenhum missing value detectado!")
```

Código-fonte 6 – missing_values (Python)

Fonte: Elaborado pelo autor (2026)

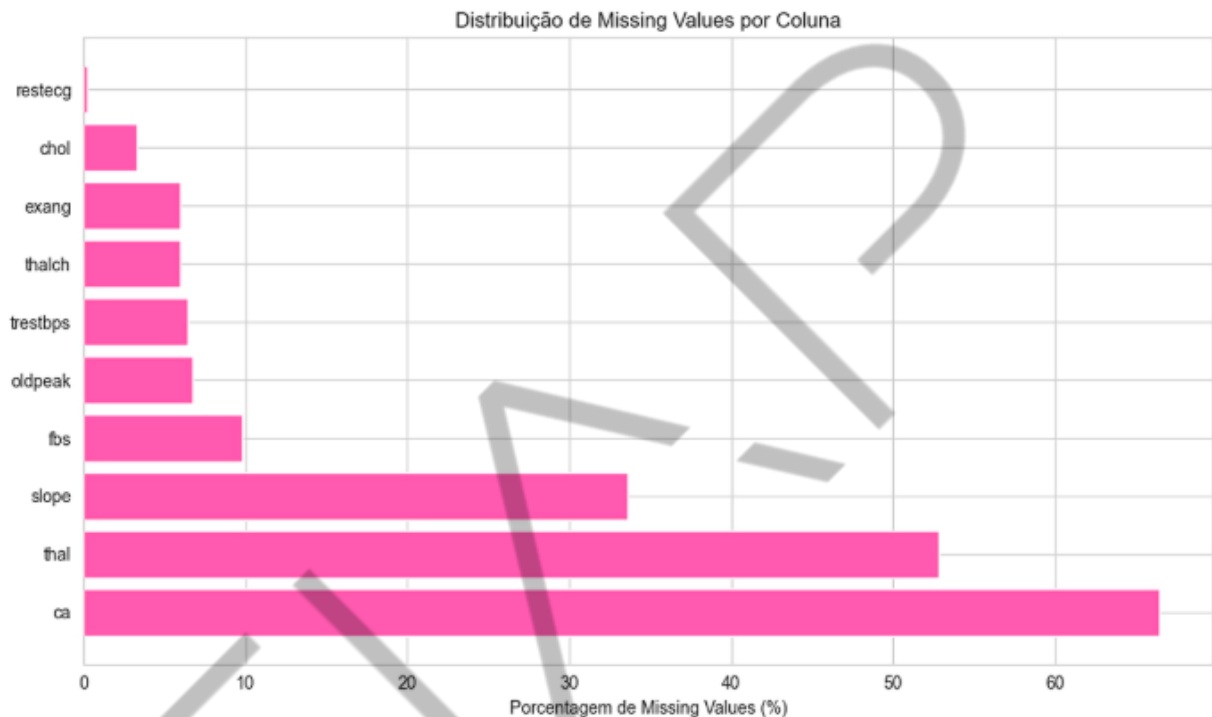


Figura 2 - Gráfico de distribuição de Missing Values por Coluna

Fonte: Elaborado pelo autor (2026)

Essa visualização gráfica complementa a tabela e é uma peça fundamental na Análise Exploratória de Dados (EDA), pois o impacto visual é mais imediato do que a leitura de dados raw.

A Análise da Variável Target é um pilar essencial na Análise Exploratória de Dados (EDA), especialmente em problemas de classificação como o nosso, que busca prever a presença ou ausência de Doença Cardíaca. Antes de treinar qualquer modelo, é crucial entender a distribuição e as características da variável que estamos tentando prever, que neste caso é a coluna num (diagnóstico de doença cardíaca).

Esta análise nos permite identificar se o nosso conjunto de dados é balanceado (i.e., se as classes "Doença Cardíaca Presente" e "Doença Cardíaca Ausente" têm

proporções semelhantes) ou desbalanceado. Para facilitar o entendimento do dataset, retomaremos a coluna num para target para facilitar a identificação do que estamos buscando prever.

Um dataset desbalanceado, onde a maioria dos exemplos pertence a uma única classe (a classe majoritária), pode levar a modelos enviesados. Por exemplo, se 90% dos pacientes não têm doença cardíaca, um modelo que simplesmente chuta "ausente" para todos os casos terá 90% de acurácia, mas será inútil na prática para identificar os pacientes doentes (a classe minoritária). Nesta subseção, vamos inspecionar a coluna target para:

- **Quantificar** a distribuição das classes (contagem de pacientes com e sem doença cardíaca).
- **Visualizar** essa distribuição, geralmente através de um gráfico para determinar o grau de balanceamento.

```
# Renomear variável `num` para `target`
df.rename(columns={'num': 'target'}, inplace=True)
# Converter target para binário (0 = sem doença, 1 = com
doença)
# No dataset original, valores > 0 indicam presença de
doença
df['target'] = (df['target'] > 0).astype(int)
print("=== DISTRIBUIÇÃO DA VARIÁVEL TARGET ===\n")
target_counts = df['target'].value_counts()
target_percentages =
df['target'].value_counts(normalize=True) * 100
print("Contagem:")
print(target_counts)
print("\nPercentual:")
for idx, pct in target_percentages.items():
    label = "Sem doença" if idx == 0 else "Com doença"
    print(f"{label} ({idx}): {pct:.2f}%")
# Visualização
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Gráfico de barras
axes[0].bar(['Sem doença', 'Com doença'],
target_counts.values, color=['lightblue', 'coral'])
axes[0].set_ylabel('Frequência')
axes[0].set_title('Distribuição da Variável Target')
axes[0].grid(axis='y', alpha=0.3)
# Gráfico de pizza
```

```
axes[1].pie(target_counts.values, labels=['Sem doença',  
'Com doença'],  
            autopct='%1.1f%%', colors=['lightblue',  
'coral'], startangle=90)  
axes[1].set_title('Proporção de Classes')  
plt.tight_layout()  
plt.show()  
# Verificar se há desbalanceamento  
ratio = target_counts.min() / target_counts.max()  
print(f"\nRatio de balanceamento: {ratio:.2f}")  
if ratio < 0.5:  
    print("⚠ Dataset desbalanceado! Considere usar técnicas  
como SMOTE ou class_weight.")  
else:  
    print("✓ Dataset razoavelmente balanceado.")
```

Código-fonte 7 – Renomear variável `num` para `target` (Python)

Fonte: Elaborado pelo autor (2026)

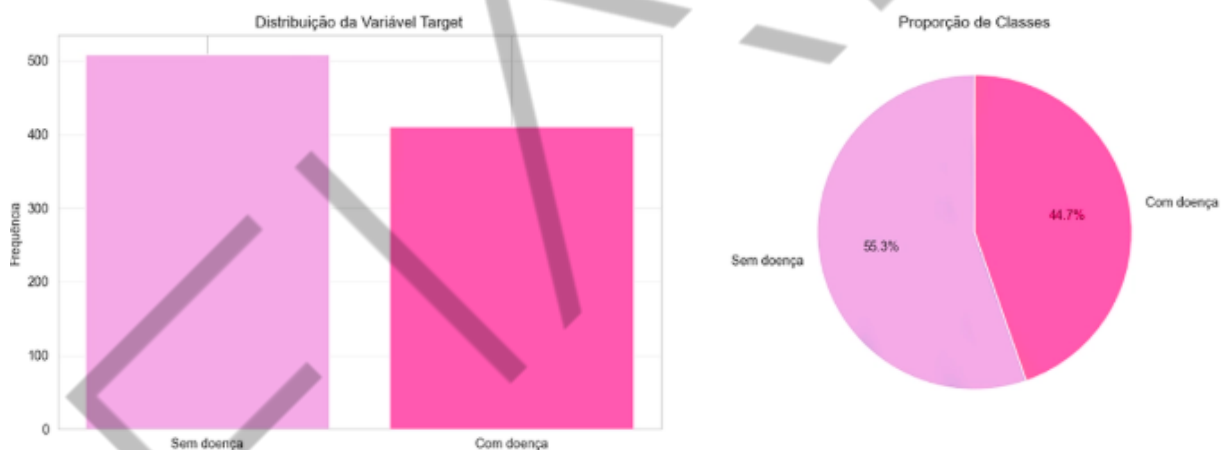


Figura 3 - Gráficos de distribuição da variável target e Proporção de Classes

Fonte: Elaborado pelo autor (2026)

A variável alvo (target) indica se há presença (1) ou ausência (0) de doença cardíaca. Na visualização, vemos 509 casos “com doença” (55,3%) e 411 “sem doença” (44,7%), o que mostra uma leve predominância da classe positiva; o ratio entre as classes é de aproximadamente 0,81, indicando um desbalanceamento pequeno. Isso significa que podemos seguir com modelos base sem grandes ajustes, mas é importante acompanhar métricas além da acurácia, como F1, precisão e recall, e observar a matriz de confusão para entender os erros.

Identificação de Outliers

A Identificação de Outliers é um passo crítico da Análise Exploratória de Dados (EDA) que visa detectar valores extremos ou anômalos que se desviam significativamente do restante do conjunto de dados. Em um contexto clínico, como o nosso dataset de Doença Cardíaca, um outlier pode ser um erro de medição ou de registro (por exemplo, um paciente com pressão arterial alta ou baixa) ou, ainda, um caso legítimo e raro que, se não tratado, pode enviesar as estatísticas e distorcer o treinamento do modelo MVP, especialmente aqueles sensíveis a grandes desvios (como a Regressão Logística).

Nesta subseção, aplicaremos abordagens estatísticas e visuais para mapear esses valores atípicos nas variáveis numéricas do nosso DataFrame. Nosso objetivo é duplo: identificar valores extremos que possam ser erros de medição e isolar casos especiais. Para isso, utilizaremos uma combinação dos métodos:

- **Z-Score:** uma métrica baseada em desvios-padrão, que marca como outliers valores que estão a mais de três desvios-padrão da média (útil para distribuições aproximadamente normais).
- **Visualização (Boxplots):** o uso de box plots (gráficos de caixa) nos permitirá uma inspeção visual rápida da dispersão dos dados e a localização imediata de pontos externos (os outliers representados fora dos "limites" do gráfico).

```
# Colunas numéricas (excluindo id e target)
numeric_cols =
df.select_dtypes(include=[np.number]).columns
numeric_cols = numeric_cols.drop(['id', 'target'])
n = len(numeric_cols)
ncols = 3
nrows = int(np.ceil(n / ncols))
fig, axes = plt.subplots(nrows, ncols, figsize=(12, 4 *
nrows))
axes = axes.ravel()
for idx, col in enumerate(numeric_cols):
    sns.boxplot(x=df[col], ax=axes[idx], color='coral')
    axes[idx].set_title(f'Boxplot: {col}',
fontweight='bold')
    axes[idx].set_xlabel(col)
    axes[idx].grid(axis='x', alpha=0.3)
```

```

col_zscore = np.abs(stats.zscore(df[col].dropna()))
outlier_count = (col_zscore > 3).sum()
axes[idx].text(0.95, 0.95, f'Outliers:
{outlier_count}',
               transform=axes[idx].transAxes,
               fontsize=9,
               verticalalignment='top',
               horizontalalignment='right',
               bbox=dict(facecolor='white', alpha=0.5,
                       edgecolor='gray'))
for ax in axes[n:]:
    fig.delaxes(ax)
plt.tight_layout()
plt.show()

```

Código-fonte 8 – Colunas numéricas (excluindo id e target) (Python)
 Fonte: Elaborado pelo autor (2026)

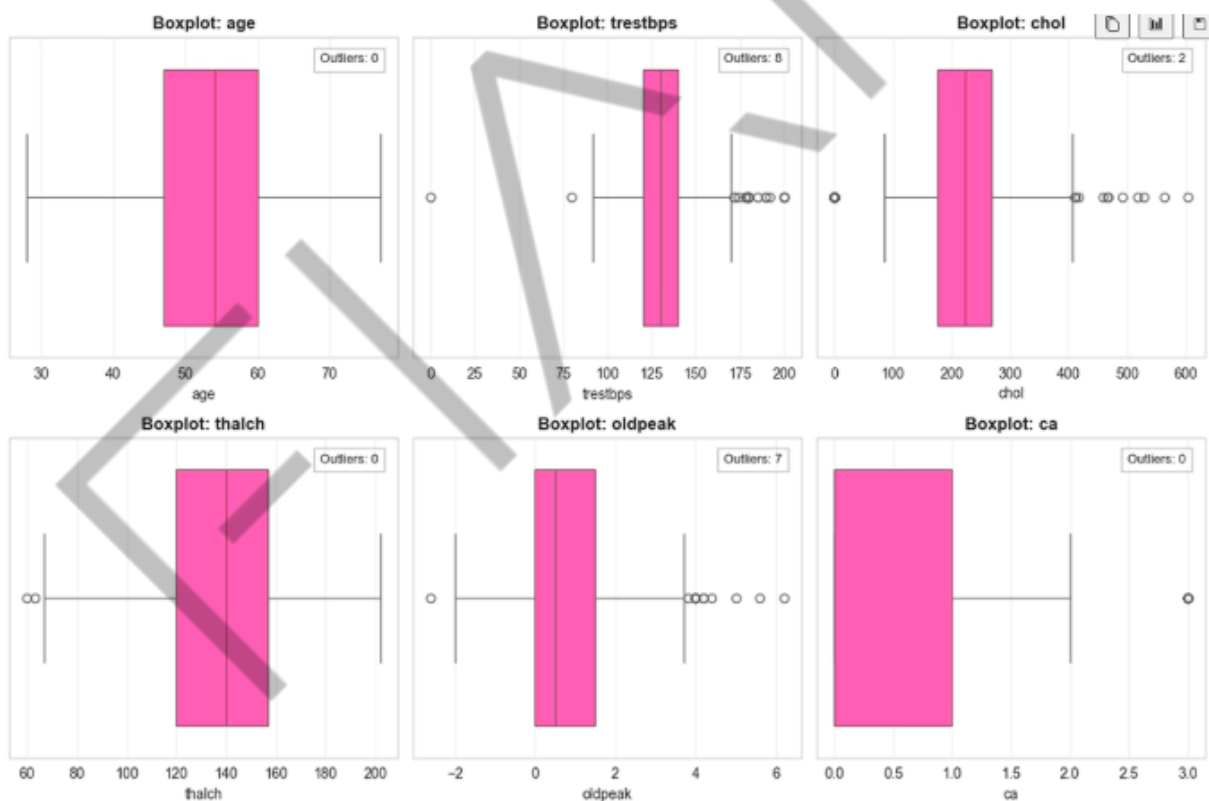


Figura 4 - Gráficos de Blox plot para análise de dispersão dos dados
 Fonte: Elaborado pelo autor (2026)

A Identificação de Outliers por meio da combinação de Boxplots e Z-Score forneceu insights valiosos sobre a distribuição e a qualidade das variáveis numéricas do dataset de Doença Cardíaca. O Boxplot permitiu uma inspeção visual imediata,

revelando pontos isolados fora dos limites que sinalizam a presença de valores atípicos. De forma complementar, a métrica do Z-Score — que quantifica o afastamento de cada ponto em relação à média, considerando-se outliers aqueles com valor absoluto superior a 3 desvios-padrão — validou estatisticamente essas observações.

Os resultados da análise indicam que a maioria das variáveis numéricas, como age, chol, thalch, e ca, apresenta uma boa distribuição, com poucos ou nenhum outlier significativo por ambos os critérios. Por exemplo, a variável ca (número de vasos principais coloridos por fluoroscopia) não registrou outliers estatísticos ($Z\text{-Score} > 3$) e sua representação visual no Boxplot foi limpa. A presença de outliers foi mais notável nas variáveis trestbps (pressão arterial em repouso) e oldpeak (depressão do ST). Porém, a contagem de casos extremos nessas colunas foi relativamente baixa.

Em conclusão, a análise sugere que o conjunto de dados possui uma qualidade inicial alta no que tange à detecção de valores extremos. O desvio dos dados é controlável. Portanto, provavelmente não será necessária uma estratégia de remoção agressiva (dropping) de linhas. Para mitigar o efeito dos poucos outliers remanescentes em trestbps e oldpeak antes da modelagem, técnicas simples de tratamento, como capping (limitando os valores extremos aos limites estatísticos definidos pelo IQR ou Z-Score), devem ser suficientes para garantir a robustez e a generalização do MVP.

De qualquer forma, a correta identificação e o subsequente tratamento (imputação, remoção ou transformação) de outliers são importantes para garantir a robustez e a generalização de modelos de classificação.

Para complementar a análise de missing values e a identificação de outliers discutidas, a Análise Exploratória de Dados (EDA) frequentemente exige a aplicação de um arsenal de outras técnicas investigativas. A inspeção dos dados é um processo iterativo e multifacetado, onde diferentes ferramentas nos ajudam a desvendar a história completa por trás do conjunto de dados de Doença Cardíaca.

Isso inclui, mas não se limita a:

- **Análise de Anomalias e Valores Inválidos:** ir além dos outliers estatísticos para buscar erros de registro, valores fora do domínio lógico (ex.: idade negativa, pressão arterial implausível) ou strings em colunas que deveriam

ser numéricas. A simples contagem de valores únicos (`df['coluna'].unique()`) pode revelar categorias que são sinônimos e precisam ser unificadas ou valores que são claramente erros.

- **Análise de Distribuições:** utilizar histogramas e gráficos de densidade para visualizar a forma, a assimetria (skewness) e a curtose (kurtosis) das variáveis numéricas. Entender a distribuição ajuda a decidir se é necessária uma transformação logarítmica ou de potência antes da modelagem para variáveis muito assimétricas, garantindo que o modelo MVP não seja distorcido.
- **Análise de Correlações:** construir heatmaps (mapas de calor) de correlação para quantificar e visualizar o relacionamento linear entre pares de variáveis. Uma alta correlação entre duas features preditoras pode indicar multicolinearidade, o que impacta negativamente modelos lineares (como a Regressão Logística que utilizaremos no nosso MVP). Inversamente, uma forte correlação entre uma feature e a variável target (Doença Cardíaca) sugere que essa feature é altamente preditiva e deve ser mantida.
- **Análise Bivariada:** explorar as relações entre as variáveis preditoras e a target de forma mais visual (ex.: boxplots da pressão arterial (trestbps) ou colesterol (chol) divididos por pacientes "Com Doença" e "Sem Doença") e estatística (ex.: tabelas de contingência para variáveis categóricas).

Todas essas abordagens fornecem insights que guiam as decisões do próximo passo de preparação e limpeza dos dados e, conseqüentemente, aumentam as chances de sucesso do nosso MVP de Modelo.

Passo 3: Preparação e Limpeza dos Dados

Tratamento de valores faltantes (missing values)

O Tratamento de Valores Faltantes (Missing Values) é uma das etapas mais críticas e obrigatórias da preparação de dados. Como identificamos, a presença de dados ausentes (representados como NaN, Null ou ?) em variáveis como `thal` e `ca` é um desafio que precisa ser resolvido antes da modelagem. A maior parte dos

algoritmos de Machine Learning não consegue processar dados incompletos. Portanto, nesta subseção, vamos implementar uma estratégia de imputação para garantir que nosso dataset esteja limpo e pronto para a fase de construção do MVP.

Nossa estratégia de tratamento será dupla e focada no contexto de dados clínicos:

- **Variáveis Categóricas e Nominais:** para colunas do tipo object (categóricas, como thal e cp), onde a moda (valor mais frequente) é uma representação robusta do dado ausente (assumindo que os dados ausentes seguem a distribuição dos dados presentes), usaremos a moda para imputação.
- **Variáveis Numéricas:** para colunas numéricas (como trestbps, chol, etc.), a mediana é frequentemente preferida em detrimento da média, pois é menos suscetível à distorção causada pela presença de outliers (que inspecionamos anteriormente).

```
print("=== TRATAMENTO DE MISSING VALUES ===\n")

# Copiar o dataframe original para preservar os dados
df_clean = df.copy()

# Imputação para variáveis numéricas: mediana
num_missing = ['trestbps', 'chol', 'thalch', 'oldpeak',
'ca']
for col in num_missing:
    if col in df_clean.columns:
        median = df_clean[col].median()
        df_clean[col].fillna(median, inplace=True)
        print(f"{col}: imputado com mediana ({median:.2f})")

# Imputação para variáveis categóricas: moda
cat_missing = ['thal', 'slope', 'fbs', 'exang', 'restecg']
for col in cat_missing:
    if col in df_clean.columns:
        mode = df_clean[col].mode()[0]
        df_clean[col].fillna(mode, inplace=True)
        print(f"{col}: imputado com moda ('{mode}')")

print("\n✓ Tratamento de missing values concluído!")
print(f"Shape após imputação: {df_clean.shape}")
print("Valores ausentes restantes por coluna:")
print(df_clean.isnull().sum())
```

Código-fonte 9 – Tratamento de Missing Values (Python)
Fonte: Elaborado pelo autor (2026)

O Tratamento de Valores Faltantes (Missing Values) foi executado com sucesso e garantiu que o DataFrame (`df_clean`) estivesse totalmente livre de dados ausentes, uma condição fundamental para a próxima etapa de modelagem preditiva. A estratégia de imputação foi aplicada de forma contextualizada, utilizando a mediana para as variáveis numéricas (como ``trestbps``, ``chol``, ``thalch``, ``oldpeak``, e ``ca``) para mitigar a influência de outliers identificados na etapa de EDA. Para as variáveis categóricas (como ``thal``, ``slope``, ``fbs``, ``exang``, e ``restecg``), a moda foi utilizada, assumindo que a categoria mais frequente é a melhor estimativa para os valores desconhecidos, preservando a distribuição original dos dados.

O resultado da inspeção final confirma que todas as 920 linhas do dataset foram preservadas e que a contagem de valores nulos em todas as colunas foi zerada. Apesar de o Tratamento de Valores Faltantes (via mediana/moda) ter preparado o DataFrame para a construção imediata do MVP, é importante saber que o processo de preparação e limpeza de dados é altamente flexível e adaptável. Em projetos mais complexos ou em iterações futuras deste modelo, outras técnicas de preparação podem ser aplicadas de forma complementar, conforme a necessidade de refinar o conjunto de dados. Isso pode incluir:

- **Tratamento de Outliers:** embora a mediana tenha mitigado o efeito dos outliers nas variáveis numéricas durante a imputação, a aplicação de técnicas mais diretas, como o capping (limitando valores extremos aos limites definidos pelo Z-Score), pode ser importante para garantir que o modelo não seja indevidamente influenciado por valores muito atípicos.
- **Transformação de Variáveis Numéricas:** variáveis com distribuições muito assimétricas (com alto skewness), como potencialmente o `chol` ou `trestbps`, podem se beneficiar de transformações logarítmicas ou de Box-Cox; estas transformações ajudam a normalizar a distribuição dos dados.
- **Feature Engineering:** a criação de novas features a partir das existentes (e.g., índices de saúde combinados, ou categorização de idades em faixas) pode adicionar poder preditivo ao modelo. Veremos mais sobre este tema na próxima aula.

- **Outras Técnicas de Codificação:** outras técnicas podem ser aplicadas dependendo dos resultados obtidos na experimentação do modelo de MVP, sendo este um processo dinâmico e iterativo.

Codificação de Variáveis Categóricas (One-Hot Encoding)

Algumas variáveis no nosso dataset são categóricas e nominais (não-ordenadas), como sex, cp, restecg, thal, e slope. Modelos de Machine Learning, como a Regressão Logística que usaremos no MVP, requerem que todas as features de entrada sejam numéricas. Nesta etapa, usaremos a técnica de One-Hot Encoding (ou criação de variáveis dummy). Essa técnica transforma cada categoria única de uma variável categórica em uma nova coluna binária (0 ou 1). Isso é feito para evitar que o modelo atribua uma ordem ou peso artificial a categorias que são intrinsecamente nominais. Por exemplo, se tivéssemos cp com categorias 'A', 'B', 'C', o modelo poderia entender que 'C' > 'B' > 'A', o que não é o caso. O One-Hot Encoding resolve isso:

```
# Remova a coluna `dataset`
df_clean.drop(columns=['dataset'], inplace=True)
# One-hot encoding das variáveis categóricas relevantes
categorical_cols = ['sex', 'cp', 'restecg', 'slope',
'thal']
df_encoded = pd.get_dummies(df_clean,
columns=categorical_cols, drop_first=True)
print(f"Shape após one-hot encoding: {df_encoded.shape}")
df_encoded.head()
```

Código-fonte 10 – One-Hot Encoding (Python)
Fonte: Elaborado pelo autor (2026)

Com esta limpeza e preparação básica concluída – que incluiu também a codificação de variáveis categóricas – o conjunto de dados está agora pronto para a fase de Construção do Modelo MVP, onde utilizaremos a Regressão Logística para estabelecer a primeira linha de base de desempenho na predição de Doença Cardíaca.

Passo 4: Construção do Modelo MVP (Minimum Viable Product)

Nosso objetivo neste momento não é a perfeição, mas validar rapidamente se os dados possuem sinal preditivo usando um modelo simples, como a Regressão Logística. Neste estágio inicial do projeto, o foco principal reside na agilidade e na

eficácia da validação. Nosso objetivo primordial é estabelecer, o mais rápido possível, se o conjunto de dados em análise contém um "sinal preditivo" útil, ou seja, se as variáveis disponíveis têm a capacidade real de prever o alvo desejado acima de um nível de chance.

Para atingir essa validação rápida e eficiente, a estratégia adotada é a construção e teste de um Produto Mínimo Viável (MVP) de Modelagem. Este MVP deve ser estruturalmente simples e de fácil interpretação. Por isso, utilizamos um modelo base, como a Regressão Logística (para problemas de classificação) ou a Regressão Linear (para problemas de regressão).

Por que a Regressão Logística é a Escolha Ideal para o MVP?

- **Simplicidade e Interpretabilidade:** a Regressão Logística é um algoritmo fundamental, cuja mecânica é transparente. Isso permite que tanto a equipe técnica quanto as partes interessadas compreendam rapidamente como as features estão influenciando o resultado, facilitando a identificação do sinal preditivo.
- **Referencial (Baseline) de Desempenho:** o desempenho do modelo simples serve como um baseline. Se mesmo um modelo simplificado como este já apresentar resultados significativos, isso é um forte indicativo de que o conjunto de dados possui um sinal robusto. Modelos mais complexos subsequentes (como Gradient Boosting ou Redes Neurais) precisarão superar este baseline para justificar sua complexidade e custo computacional.
- **Ciclos de Feedback Rápidos:** a facilidade e rapidez com que a Regressão Logística pode ser treinada e avaliada garantem ciclos curtos de experimentação (o conceito de fail fast ou succeed fast). Se o modelo falhar em capturar um sinal, aprendemos rapidamente e podemos ajustar a engenharia de features ou a seleção de dados sem ter investido tempo excessivo em modelos mais custosos.

Em resumo, a fase inicial é um exercício de validação de hipóteses (data validation), onde a velocidade e a clareza sobre a existência do sinal preditivo superam a busca por métricas de desempenho otimizadas. A perfeição virá apenas após a

confirmação de que os dados fundamentam, de fato, o problema de predição proposto.

Para a construção do modelo de baseline precisamos dividir o dataset pré-processado em conjuntos de treino e teste. Utilizamos a biblioteca sklearn com o método `train_test_split` para isso (definiremos que 20% do dataset será utilizado como conjunto de teste e os 80% restantes para treino do modelo):

```
from sklearn.model_selection import train_test_split

# Divida o dataset em features (X) e target (y)
X = df_encoded.drop(columns=['id', 'target'])
y = df_encoded['target']

# Divida em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)
```

Código-fonte 11 – Método `train_test_split` (Python)
Fonte: Elaborado pelo autor (2026)

O `random_state` permite garantir a reprodutibilidade do experimento, isto é, sempre que executarmos novamente o experimento, os dados serão divididos com as mesmas amostras de dados.

Para assegurar um processo de experimentação robusto, rastreável e que permita a reprodução dos resultados de Machine Learning, será introduzido o MLflow. Este framework open-source gerencia o ciclo de vida de ML através de quatro componentes principais: MLflow Tracking (para registrar e comparar parâmetros e métricas), MLflow Projects (para empacotar código de forma reprodutível), MLflow Models (para padronizar modelos) e MLflow Model Registry (para gerenciar modelos em produção).

O foco inicial será o MLflow Tracking, que permitirá registrar detalhadamente os parâmetros e métricas do nosso Produto Mínimo Viável (MVP), criando um registro ("Run") que facilitará a documentação do experimento e a comparação objetiva de futuras iterações e modelos mais complexos, garantindo que a avaliação do MVP seja devidamente catalogada no próximo passo. Nas próximas aulas conheceremos mais profundamente o uso do MLFlow para experimentação.

Finalmente, coletaremos e analisaremos métricas cruciais de desempenho, tais como acurácia (a proporção de previsões corretas em relação ao total de casos), o `f1_score` (a média harmônica da precisão e do recall), a `precision` (o quão confiável é o modelo ao prever a classe positiva, minimizando falsos positivos) e o `recall` (a capacidade do modelo de encontrar todas as amostras positivas, minimizando falsos negativos). A avaliação cuidadosa dessas métricas é essencial para compreender as forças e fraquezas do nosso Modelo de Valor Mínimo Viável (MVP) e determinar se ele está pronto para a próxima fase de experimentação ou se requer ajustes e otimizações adicionais. Essa análise de desempenho nos guiará na iteração e melhoria contínua do modelo.

```
import mlflow
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score,
precision_score, recall_score
mlflow.set_experiment("heart_disease_classification")
with
mlflow.start_run(run_name="logistic_regression_baseline"):
    model = LogisticRegression(random_state=42)
    model.fit(X_train, y_train)
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)
    train_accuracy = accuracy_score(y_train, y_pred_train)
    test_accuracy = accuracy_score(y_test, y_pred_test)
    test_f1 = f1_score(y_test, y_pred_test)
    test_precision = precision_score(y_test, y_pred_test)
    test_recall = recall_score(y_test, y_pred_test)
    mlflow.log_metric("train_accuracy", train_accuracy)
    mlflow.log_metric("test_accuracy", test_accuracy)
    mlflow.log_metric("test_f1_score", test_f1)
    mlflow.log_metric("test_precision", test_precision)
    mlflow.log_metric("test_recall", test_recall)
    overfitting = train_accuracy - test_accuracy
    print(f"=== LOGISTIC REGRESSION ===")
    print(f"Train Accuracy: {train_accuracy:.4f}")
    print(f"Test Accuracy: {test_accuracy:.4f}")
    print(f"Test F1 Score: {test_f1:.4f}")
    print(f"Test Precision: {test_precision:.4f}")
    print(f"Test Recall: {test_recall:.4f}")
    print(f"Overfitting: {overfitting:.4f}")
```

Código-fonte 12 – import mlflow (Python)
Fonte: Elaborado pelo autor (2026)

O experimento do MVP (Produto Mínimo Viável) de Regressão Logística para classificação de Doença Cardíaca obteve o seguinte baseline:

MÉTRICA	RESULTADO
Train Accuracy	0.82
Test Accuracy	0.80
Test F1 Score	0.83
Test Precision	0.85
Test Recall	0.80

Tabela 1 - Métricas do Modelo (Acurácia, F1-Score, Precisão e Recall)
Fonte: Elaborado pelo autor (2026)

Conclusões Principais:

- **Viabilidade Comprovada:** a Acurácia de Teste de 80.43% é significativamente superior ao baseline de chance (55.3% de acurácia se apenas chutasse a classe majoritária). Isso valida que os dados contêm um sinal preditivo robusto para a condição cardíaca.
- **Excelente Generalização (Sem Overfitting):** a diferença de apenas 2.04 pontos percentuais entre a Acurácia de Treino (82.47%) e a de Teste (80.43%) demonstra que o modelo generaliza bem para dados não vistos, indicando baixo risco de sobreajuste (overfitting).
- **Desempenho Equilibrado e Crítico (F1 Score):** o F1 Score de 83.02% comprova um ótimo equilíbrio entre as métricas críticas de saúde:

- **Precision (85.44%):** alta taxa de acerto nas previsões positivas, minimizando Falsos Positivos (pacientes saudáveis classificados como doentes).
- **Recall (80.73%):** boa capacidade de identificar os casos reais da doença, minimizando Falsos Negativos (pacientes doentes classificados como saudáveis).

Próximos Passos: o MVP estabeleceu um baseline de 80.43% (Acurácia) e 83.02% (F1 Score) para futuros modelos. O projeto deve avançar para a Modelagem Avançada, focando em:

- **Aprimoramento de Features:** refinamento do desempenho via Feature Engineering e Feature Selection.
- **Otimização do Modelo:** testar algoritmos mais complexos (Random Forest, Gradient Boosting) que deverão, obrigatoriamente, superar o desempenho deste baseline.

Em suma, a experimentação inicial foi um sucesso e forneceu a base sólida e interpretável necessária para as próximas iterações do projeto.

Data Understanding no contexto CRISP-DM

Em projetos de ciência de dados, é fundamental seguir um processo estruturado para minimizar riscos e desconhecidos. Conforme vimos, um dos frameworks mais tradicionais é o CRISP-DM (Cross-Industry Standard Process for Data Mining), que define fases claras: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation e Deployment. Após compreender os objetivos de negócio, entramos na fase de Entendimento dos Dados (Data Understanding), que foi o foco desta aula.

O propósito principal dessa fase é obter uma visão geral das fontes de dados relevantes, suas características e qualidade. Isso inclui desde coletar e integrar os dados brutos disponíveis, passando por descrever os dados (estruturas, tipos, volumes, campos disponíveis), até efetivamente explorar os dados em busca de

insights iniciais e verificar a qualidade. Em suma, trata-se de mapear tudo que temos em mãos e avaliar se esses dados são adequados para atingir os objetivos definidos.

Na prática, o Data Understanding envolve algumas tarefas essenciais. Uma referência útil as organiza em quatro passos:

- **Coletar dados iniciais** – reunir as fontes de dados disponíveis e documentar seu conteúdo. Aqui é feito um mapeamento dos dados: descrição de cada atributo (nome, significado, tipo, unidade), volume de registros e como esses dados foram coletados. É comum elaborar um dicionário de dados ou relatório descritivo que funcione como guia para todos os envolvidos no projeto.
- **Descrever os dados** – gerar estatísticas descritivas básicas. Para variáveis numéricas: médias, medianas, mínimas, máximas, quartis e desvio-padrão. Para variáveis categóricas: contagens de categorias e cardinalidade (número de valores únicos). Essa descrição ajuda a ter um panorama do conjunto de dados e a identificar possíveis surpresas (ex.: valores fora da faixa esperada, ou categorias raras inesperadas).
- **Explorar os dados (EDA)** – realizar análises exploratórias mais aprofundadas para entender distribuições e relacionamentos. Isso inclui plotar histogramas, box plots, gráficos de dispersão, tabelas cruzadas etc., para detectar padrões, tendências, correlações e outliers. A EDA é, em essência, uma etapa investigativa: formulamos perguntas e usamos os dados para respondê-las de forma visual e estatística, sem ainda construir modelos preditivos formais.
- **Verificar qualidade dos dados** – avaliar quão confiáveis e utilizáveis os dados brutos são. Nesta tarefa, buscamos identificar problemas de qualidade, como dados faltantes, valores inconsistentes, duplicatas, outliers e noise (ruído). Aplicamos aqui o princípio GIGO (Garbage In, Garbage Out): se a entrada de dados for ruim, o resultado da análise também será ruim. Portanto, documentamos todas as anomalias encontradas e já pensamos em estratégias para corrigi-las ou mitigá-las. Em projetos reais, essa etapa muitas vezes envolve comunicar-se com engenheiros(as) de

dados ou responsáveis pelas fontes, caso sejam detectados erros de extração ou registro que possam ser sanados na origem.

É importante destacar que o Data Understanding é iterativo. Embora ocorra logo após o entendimento do negócio e antes da preparação mais pesada dos dados, frequentemente descobrimos coisas no meio da modelagem que nos fazem revisitar a fase de entendimento. Assim, apesar do CRISP-DM sugerir fases lineares, na prática há um vai-e-volta constante: entendemos os dados, preparamos, modelamos e se algo estranho surgir na modelagem, voltamos para entender melhor e preparar de novo. Esse ciclo iterativo faz parte da experimentação.

Análise Exploratória de Dados (EDA) e variáveis dos dados

Como vimos, a Análise Exploratória de Dados (EDA) é parte integrante do Data Understanding. Mas vale aprofundar algumas técnicas e considerações específicas dessa etapa. A EDA serve para deixar os dados “contarem sua história” antes de impormos qualquer modelo ou hipótese rígida. Frequentemente, é através dela que cientistas de dados validam se estão fazendo as perguntas certas sobre o negócio e os dados. A EDA nos ajuda a determinar a melhor maneira de manipular as fontes de dados para obter respostas, revelando quais variáveis parecem mais promissoras, quais transformações podem ser necessárias e se existem relações óbvias que devem ser levadas em conta.

É útil categorizar as análises exploratórias conforme o tipo e quantidade de variáveis envolvidas:

- **Univariada:** análise de uma variável de cada vez. Pode ser não-gráfica (apenas estatísticas como média, mediana, desvios) ou gráfica (histogramas, box plots, gráficos de densidade). O objetivo aqui é entender a distribuição dessa variável individualmente.
- **Bivariada:** análise de duas variáveis simultaneamente, geralmente para avaliar o relacionamento entre elas. Pode ser não-gráfica (tabelas de contingência, coeficientes de correlação) ou gráfica (gráfico de dispersão para duas variáveis numéricas; box plot comparando uma variável numérica entre categorias de uma variável categórica; gráfico de barras comparando proporções de uma categórica condicionadas a outra etc.).

- **Multivariada:** análise envolvendo três ou mais variáveis simultaneamente. Pode exigir técnicas mais sofisticadas de visualização (como gráficos 3D, heatmaps de correlação de matrizes de variáveis, paralelogramos, gráfico de pares (pair plot), e até redução de dimensionalidade como PCA para projetar dados de muitas variáveis em 2D).

Ao executar EDA, é importante lembrar que ela não substitui análises estatísticas formais ou modelagem preditiva, mas serve de base para elas. Durante a EDA, consideramos os diferentes tipos de variáveis presentes e aplicamos técnicas adequadas a cada tipo:

- Variáveis numéricas (contínuas ou discretas): calcular estatísticas (média, mediana etc.), visualizar distribuição com histogramas ou kernel density plots, identificar outliers com boxplots ou z-scores.
- Variáveis categóricas: inspecionamos frequências de cada categoria e muitas vezes visualizamos essas frequências em gráficos de barra. Também é útil examinar a relação de categorias com a variável resposta.
- Variáveis textuais ou identificadores: em datasets de projetos reais, colunas como texto livre (descrições, comentários), identificadores únicos (IDs, códigos de transação) e variáveis alfanuméricas (números de documentos, códigos de produtos que misturam letras e números) requerem tratamento especial. Frequentemente, colunas com muitos valores únicos e baixa contribuição preditiva (como identificadores) são descartadas ou precisam ser submetidas a uma Engenharia de Features (Feature Engineering) complexa.

Um ponto importante na EDA é a identificação de outliers e dados anômalos. Outliers são pontos que se afastam drasticamente do restante dos dados. Eles podem surgir por diversos motivos: erros de medição ou registro, erros de digitação ou podem ser valores reais mas provenientes de uma distribuição de cauda longa. Por exemplo, se na coluna de idade houvesse alguém registrado com 200 anos, é quase certo que seria um erro de digitação (um “20.0” talvez digitado como “200”).

Depois de identificar outliers, vem a decisão de como tratá-los (parte da preparação de dados):

- Podemos remover outliers se houver evidências de que são erros ou aberrações que não nos interessam. Mas isso deve ser feito com cuidado – remover pode eliminar informações valiosas se o outlier for legítimo.
- Podemos transformar os dados para reduzir o peso dos outliers, por exemplo aplicando log nas variáveis assimétricas. Isso comprime a escala e faz outliers parecerem menos extremos na nova escala.
- Ou ainda, podemos substituir o outlier por um valor menos extremo (como a mediana), embora isso também possa distorcer um pouco a distribuição.

Outro problema de qualidade são os dados faltantes (missing data). Encontrar valores nulos ou ausentes é muito comum (“NA”, “Null” ou simplesmente campos vazios). Esses valores faltantes podem ocorrer por inúmeras razões: falhas na coleta, erros de registro ou ausência genuína de informação (por exemplo, muitos passageiros não tiveram a cabine anotada, possivelmente porque estavam em cabines compartilhadas sem número ou dormindo no convés). É crucial quantificar e tratar os missings, porque dados incompletos podem levar a resultados enviesados ou falhas no modelo. As principais estratégias para lidar com missing incluem:

- **Remoção** dos registros incompletos: simples de implementar (dropna() no pandas), porém arriscado se muitos dados forem descartados. Só é viável quando a porcentagem de missings é muito pequena e distribuída aleatoriamente.
- **Imputação simples**: preencher o valor faltante com alguma estimativa como média, mediana ou moda. Em muitos casos, é aceitável como primeira aproximação, mas deve-se notar o possível viés introduzido.
- **Imputação por modelo (ou avançada)**: usar métodos mais sofisticados como algoritmos k-NN para imputar ou até treinar um modelo de regressão para prever o valor ausente a partir de outras variáveis.
- **Análise de sensibilidade**: qualquer que seja a estratégia de imputação, é importante verificar como ela impacta o modelo. Muitas vezes testamos o modelo com e sem imputação ou com diferentes técnicas para ver se os

resultados mudam muito. Se mudarem, o tratamento dos missings está influenciando demais – possivelmente precisamos rever a abordagem.

Vale ressaltar a importância de documentar todo esse entendimento e tratamentos iniciais. Um(a) cientista de dados profissional normalmente produz relatórios de EDA, aponta problemas de qualidade e justifica as ações tomadas (remover ou imputar, transformar variáveis etc.). Essa documentação garante que a equipe e stakeholders compreendam as limitações dos dados e as premissas adotadas. E lembra: caso essa fase seja negligenciada, todo o projeto pode ser comprometido. Investir tempo em EDA e limpeza inicial é investir na base do projeto.

Construção de um MVP de modelo e experimentação rápida

Com os dados explorados e minimamente limpos, entra em cena o conceito de MVP de modelos. MVP (Minimum Viable Product – Produto Mínimo Viável) é um termo originado no desenvolvimento de startups de software, popularizado por Eric Ries. Em essência, significa criar a versão mais simples possível de um produto que ainda entregue valor e possibilite aprendizado, para então iterar e melhorá-lo. No contexto de Ciência de Dados, o MVP se traduz em desenvolver rapidamente um modelo funcional, ainda que simples, para validar premissas e colher feedback real.

Por que isso é importante? Em projetos de ML, é tentador gastar muito tempo tentando construir a solução perfeita de primeira – dados perfeitamente limpos, modelo ultra sofisticado etc. Porém, isso é arriscado: podemos descobrir tarde demais que o problema era mal definido ou que os dados não suportam uma solução acurada. Construir um modelo inicial simples (por exemplo, um modelo linear ou de árvore de decisão com poucas variáveis) atende a dois propósitos principais:

- **Validação rápida da viabilidade:** o modelo MVP permite coletar o máximo de aprendizado validado sobre o desempenho do modelo no mundo real com mínimo esforço. Se esse modelo inicial já mostra resultados promissores (melhor que aleatório, tendência correta), ganhamos confiança de que vale a pena investir em melhorias. Se ele falha completamente, isso acende uma luz amarela – talvez os dados não contêm sinal suficiente ou o problema precisa ser reformulado.

- **Feedback e iteração:** com um MVP em mãos, podemos apresentá-lo a stakeholders ou testá-lo em um ambiente de sandbox para obter feedback. Às vezes, só ao ver um modelo funcionando (mesmo que rudimentar), especialistas de domínio conseguem opinar sobre quais variáveis são relevantes ou se o comportamento faz sentido. Além disso, colocar um modelo cedo em produção controlada (por exemplo, para alguns usuários ou em testes A/B) permite medir impacto real e evitar surpresas posteriores. Uma máxima em projetos de IA modernos é "se é para falhar, que falhe rápido e barato" – melhor descobrir limitações no MVP do que depois de meses de trabalho.

No caso de modelos preditivos, um MVP pode ser tão simples quanto um modelo de regressão linear ou um classificador baseline. Por exemplo, se queremos prever a rotatividade de clientes (churn), um MVP pode ser um modelo de regressão logística usando 2 ou 3 variáveis principais que já existem, sem esperar por meses para coletar dados novos ou desenvolver features complexas. Isso coloca um modelo funcional rapidamente na mão de tomadores de decisão, mesmo que com acurácia modesta. Com os resultados desse MVP, a equipe pode iterar: coletar mais dados, incluir mais variáveis, testar algoritmos mais avançados etc. guiados pelo que aprenderam inicialmente.

Uma questão prática é: como conciliar o MVP rápido com as boas práticas de engenharia e qualidade? Afinal, um modelo improvisado pode não escalar ou não se integrar bem ao sistema final. A indústria tech tem abordado isso investindo em plataformas de experimentação e automação de ML. Por exemplo, o MLflow, um projeto de código aberto iniciado pela Databricks, permite a cientistas de dados gerenciar o ciclo de vida de ML de ponta a ponta. Ele oferece componentes para rastrear experimentos (Tracking), empacotar código (Projects) e gerenciar modelos (Models), o que reduz drasticamente o custo e o tempo de colocar novos modelos em produção, através de um sistema unificado para lidar com artefatos e reprodutibilidade.

A Databricks, plataforma líder em Data and AI, fornece a infraestrutura que integra o MLflow nativamente, permitindo que a criação de MVPs e a transição para a produção aconteçam de forma mais rápida e escalável. Na prática, isso significa que data scientists podem criar protótipos (MVPs) e testá-los em produção muito mais

rápido do que se tivessem que cuidar manualmente de banco de dados, ETLs e monitoramento. Essa tendência de Ferramentas de MLOps reflete uma demanda do mercado: encurtar o ciclo entre ideia e modelo implantado para colher valor mais cedo e ajustar o rumo com agilidade.

Resumindo, a construção de um MVP de modelo é a aplicação da filosofia lean startup à ciência de dados. Envolve entregar valor inicial rápido e aceitar que o primeiro modelo não será perfeito, mas usá-lo como base para aprender e melhorar continuamente. Isso requer uma mentalidade de experimentação contínua, onde cada iteração do modelo incorpora novos aprendizados dos dados e do feedback real.

Em projetos de IA de ponta, inclusive, já se defende o conceito de Minimum Viable Intelligence (MVI) – isto é, não esperar ter o melhor algoritmo para começar a gerar inteligência a partir dos dados; comece com algo viável e aprenda com isso. Empresas que adotam esse ritmo iterativo conseguem ajustar seus produtos de dados de forma incremental e rápida, saindo na frente da concorrência que porventura fique paralisada planejando a solução perfeita.

MERCADO, CASES E TENDÊNCIAS

Para ilustrar como todos esses conceitos se unem em cenários reais, vejamos um exemplo de aplicação de EDA e MVP de modelos no mercado: previsão de preços e demanda na Airbnb. A Airbnb valoriza a experimentação rápida em seus projetos de dados. Em um caso divulgado no Airbnb Tech Blog, a equipe de ciência de dados buscou criar um modelo para prever o Lifetime Value (LTV) de novos anúncios na plataforma. Antes de chegar a um modelo final de Machine Learning sofisticado, eles seguiram etapas parecidas com as que estudamos:

- **Entendimento dos dados e EDA:** cientistas de dados analisaram dados históricos de reservas e avaliaram quais features poderiam influenciar o valor de um anúncio (localização, tipo de acomodação, preço relativo aos similares, taxa de ocupação etc.). Utilizando EDA, identificaram correlações e padrões – por exemplo, observaram que a porcentagem de noites disponíveis nos próximos 6 meses é um indicador importante (um anfitrião que deixa muitos dias disponíveis talvez tenha menor demanda, refletindo em menor LTV).
- **Qualidade e preparação:** validaram valores ausentes e consistência. Em casos como esse, dados de usuários podem ter campos nulos; supõe-se que fizeram imputações ou descartes quando necessário. Também transformaram variáveis categóricas e numéricas (por exemplo, usaram one-hot encoding para atributos como tipo de quarto ou bairro).
- **MVP de modelo (prototipagem):** com os dados preparados, inicialmente treinaram modelos simples usando bibliotecas em Python. Esse protótipo permitiu avaliar rapidamente quais features mais contribuíram e a performance obtida. A equipe conseguia iterar trocando algoritmos e ajustando parâmetros localmente de forma ágil.
- **Validação e iteração:** uma vez satisfeito com um modelo candidato, graças à infraestrutura de ML interna, eles conseguiram colocá-lo em produção experimental e medir seu impacto sem ter que reescrever todo o código. Descobertas interessantes vieram à tona apenas após o lançamento do modelo inicial, levando a iterações. Um caso narrado foi de um modelo de

deep learning de recomendação que a Airbnb lançou e inicialmente apresentou problemas – somente com o modelo em produção e observando as métricas em tempo real eles perceberam a necessidade de otimizações adicionais. Isso reforça a ideia: o aprendizado verdadeiro ocorre quando expomos o modelo ao mundo real.

EMVSP

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, revisamos e praticamos as etapas iniciais cruciais em projetos de Machine Learning. Primeiramente, discutimos como realizar o mapeamento e entendimento dos dados brutos, identificando a estrutura do dataset e as características de cada variável. Em seguida, mergulhamos na Análise Exploratória de Dados (EDA) – uma investigação sistemática para extrair insights iniciais e detectar padrões, tendências ou anomalias nos dados.

Vimos na prática, com o caso de Doenças Cardíacas (Heart Disease), como a EDA revela distribuições e relacionamentos importantes entre os dados. Identificamos também problemas de qualidade de dados que precisam ser enfrentados antes da modelagem: destacamos a detecção de outliers, entendendo que valores muito fora da curva podem distorcer resultados, e a presença de dados faltantes, que exigem decisões de imputação ou remoção. Abordamos diversas técnicas iniciais de limpeza e transformação, preenchimento de valores ausentes com medidas estatísticas simples e transformação de variáveis categóricas (codificação) para viabilizar sua utilização em modelos. Discutimos que essa etapa de preparação básica muitas vezes já melhora a qualidade do dataset e, portanto, a performance dos modelos – lembrando sempre do princípio garbage in, garbage out.

Um conceito central foi o de Experimentação e MVP de Modelos. Aprendemos que, em vez de buscar de imediato a solução perfeita, é muito valioso construir um modelo mínimo viável rapidamente, após a EDA inicial. Esse MVP de modelo serve para validar a viabilidade do problema e do approach, oferecendo uma linha de base de desempenho. Isso não apenas economiza tempo e recursos, evitando investir pesado em ideias não testadas, como também gera aprendizados práticos mais rápido.

Na seção de mercado e casos, vimos exemplos concretos, como a estratégia da Airbnb de integrar análise exploratória rigorosa com ciclos rápidos de prototipagem de modelos. Este caso evidencia tendências atuais, incluindo o uso crescente de plataformas e ferramentas automatizadas para agilizar do insight inicial até o modelo em produção.

Por fim, reforçamos que todas as etapas abordadas – mapeamento dos dados, EDA, tratamento de outliers e missing e criação de um modelo inicial – trabalham em sinergia para construir uma base sólida para qualquer projeto de Machine Learning. Com os conceitos e técnicas desta aula, você está apto a iniciar um projeto de data science com o pé direito: conhecendo bem seus dados e validando cedo suas ideias de modelagem. Assim, evitam-se surpresas desagradáveis mais adiante e aumentam-se as chances de sucesso do modelo final.



REFERÊNCIAS

CHANG, R. **Using Machine Learning to Predict Value of Homes On Airbnb**. 2017. Disponível em: <https://medium.com/airbnb-engineering/using-machine-learning-to-predict-value-of-homes-on-airbnb-9272d3d4739d>. Acesso em: 28 jan. 2026.

CONCEITOS TECH. **Entenda os Outliers: Identificação e Tratamento em Dados**. 2023. Disponível em: <https://conceitos.tech/tutoriais/inteligencia-artificial/machine-learning/o-que-sao-outliers-e-como-identifica-los-nos-dados/>. Acesso em: 28 jan. 2026.

CONCEITOS TECH. **Estratégias para Gerenciar Dados Faltantes em Machine Learning**. 2023. Disponível em: <https://conceitos.tech/tutoriais/inteligencia-artificial/machine-learning/como-lidar-com-dados-faltantes-em-machine-learning>. Acesso em: 28 jan. 2026.

GARCIA, J. P. **What Airbnb discovered after launching its first AI product?** 2023. Disponível em: <https://joseparreogarcia.substack.com/p/what-airbnb-discovered-after-launching>. Acesso em: 28 jan. 2026.

IBM ANALYTICS. **O que é análise exploratória de dados (EDA)?** 2026. Disponível em: <https://www.ibm.com/br-pt/think/topics/exploratory-data-analysis>. Acesso em: 28 jan. 2026.

ILUMEO. **O Produto Mínimo Viável – MVP – para Dados**. 2021. Disponível em: <https://ilumeo.com.br/categorias/2021-02-09-o-produto-minimo-viavel-mvp-para-dados>. Acesso em: 28 jan. 2026.

KANOKI; DATA SCIENCE PM. **What is a Data Science MVP?** 2020. Disponível em: <https://www.datascience-pm.com/data-science-mvp>. Acesso em: 28 jan. 2026.

MIOT, H. A. **Valores anômalos e dados faltantes em estudos clínicos e experimentais**. Jornal Vascular Brasileiro, v. 18, n. 3, 2019. Disponível em: https://www.researchgate.net/publication/333487764_Valores_anomalos_e_dados_faltantes_em_estudos_clinicos_e_experimentais. Acesso em: 28 jan. 2026.

RICHARDSON. **A Fase de Business Data Understanding (Compreensão dos Dados do Negócio)**. 2023. Disponível em: https://dev.to/_richardson_/a-fase-de-business-data-understanding-compreensao-dos-dados-do-negocio-2bab. Acesso em: 28 jan. 2026.

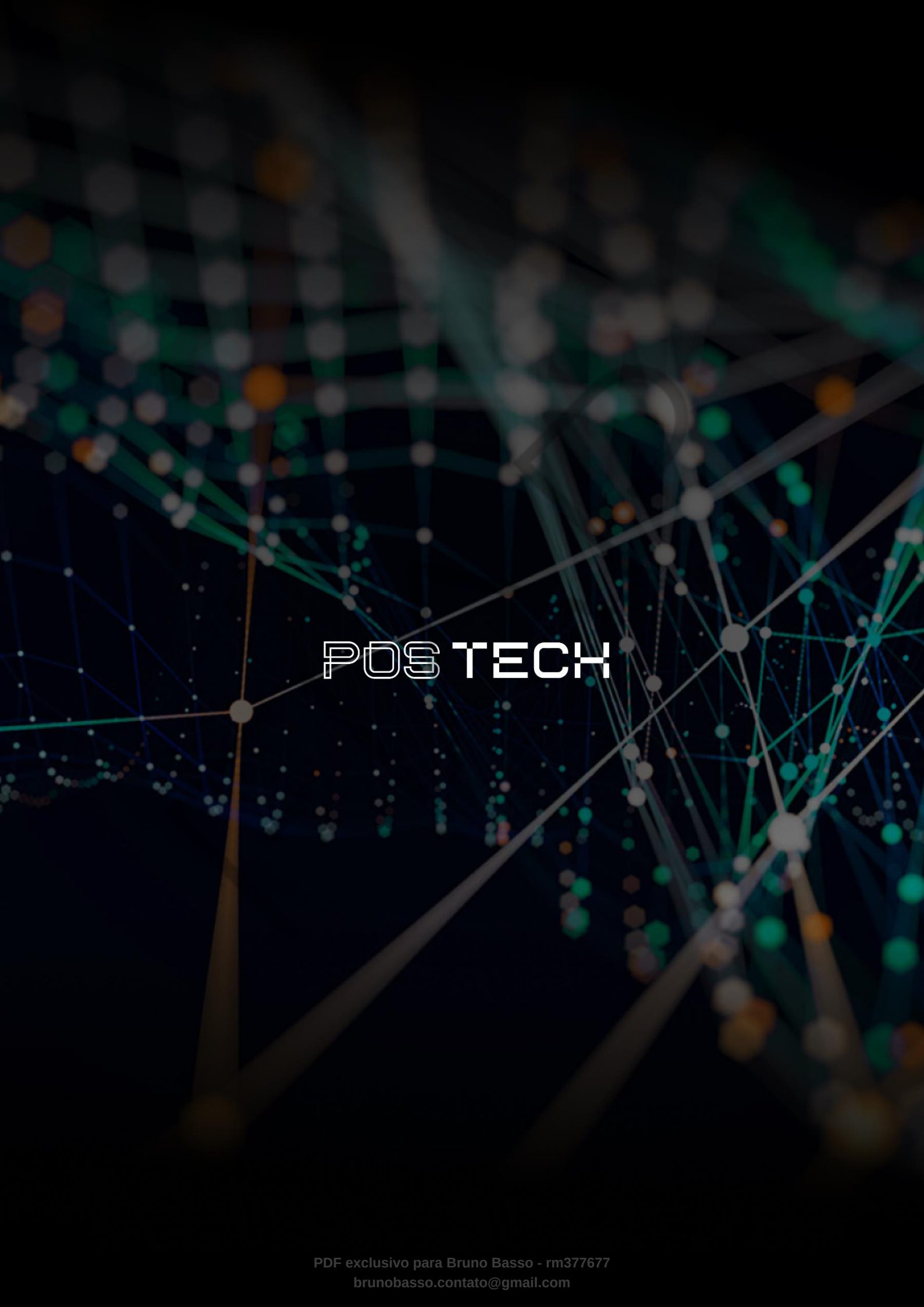
SONY, R. K. **Heart Disease Data**. 2023. Disponível em: <https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data/data>. Acesso em: 28 jan. 2026.

UCI MACHINE LEARNING REPOSITORY. **Heart Disease**. 1988. Disponível em: <https://archive.ics.uci.edu/dataset/45/heart+disease>. Acesso em: 28 jan. 2026.

PALAVRAS-CHAVE

EDA. MVP. CRISP-DM.

CRISP-DM



POSTECH